In [8]:
```
import opendatasets as od
```

In [9]:
```
od.download("https://www.kaggle.com/datasets/thebumpkin/auto-insurance-claims-updated-to-2
```

```
Skipping, found downloaded files in "./auto-insurance-claims-updated-to-2024" (use force=T
rue to force download)
```

In [10]:
```
df = pd.read_csv("/Users/brandonphan1/Downloads/AutoInsuranceClaims2024.csv")
df.head()
```

Out[10]:

	Customer	State	Customer Lifetime Value	Response	Coverage	Coverage Index	Education	Education Index	Effective To Date	Employment Status
0	QC35222	California	3622.69	No	Basic	0	Bachelor	2	1/1/2024	Employed
1	AE98193	Washington	10610.21	No	Basic	0	High School or Below	0	1/1/2024	Unemployed
2	TM23514	Oregon	13868.02	No	Extended	1	College	1	1/1/2024	Employed
3	WB38524	California	4008.95	No	Basic	0	High School or Below	0	1/1/2024	Employed
4	QZ42725	Washington	3119.69	No	Basic	0	Bachelor	2	1/1/2024	Unemployed

5 rows × 34 columns

In [11]:
```
df.columns
```

Out[11]:
```
Index(['Customer', 'State', 'Customer Lifetime Value', 'Response', 'Coverage',
       'Coverage Index', 'Education', 'Education Index', 'Effective To Date',
       'Employment Status', 'Employment Status Index', 'Gender', 'Income',
       'Location', 'Location Index', 'Marital Status', 'Marital Status Index',
       'Monthly Premium Auto', 'Months Since Last Claim',
       'Months Since Policy Inception', 'Number of Open Complaints',
       'Number of Policies', 'Policy Type', 'Policy Type Index', 'Policy',
       'Policy Index', 'Renew Offer Type', 'Sales Channel',
       'Sales Channel Index', 'Total Claim Amount', 'Vehicle Class',
       'Vehicle Class Index', 'Vehicle Size', 'Vehicle Size Index'],
      dtype='object')
```

In [12]:
```
df.iloc[0]
```

Out[12]:
```
Customer                                QC35222
State                                California
Customer Lifetime Value                 3622.69
Response                                     No
Coverage                                  Basic
Coverage Index                                0
Education                              Bachelor
Education Index                               2
Effective To Date                       1/1/2024
Employment Status                       Employed
Employment Status Index                       1
```

Loading [MathJax]/extensions/Safe.js

```
Gender                                       F
Income                                   65163
Location                                 Urban
Location Index                               2
Marital Status                         Married
Marital Status Index                         1
Monthly Premium Auto                        93
Months Since Last Claim                      4
Months Since Policy Inception              107
Number of Open Complaints                    3
Number of Policies                           1
Policy Type                     Corporate Auto
Policy Type Index                            1
Policy                            Corporate L2
Policy Index                                 4
Renew Offer Type                             3
Sales Channel                              Web
Sales Channel Index                          0
Total Claim Amount                       380.9
Vehicle Class                     Four-Door Car
Vehicle Class Index                          1
Vehicle Size                           Medsize
Vehicle Size Index                           1
Name: 0, dtype: object
```

In [13]:

```
df.shape
```

Out[13]: (9134, 34)

In [14]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9134 entries, 0 to 9133
Data columns (total 34 columns):
 #   Column                         Non-Null Count  Dtype
---  ------                         --------------  -----
 0   Customer                       9134 non-null   object
 1   State                          9134 non-null   object
 2   Customer Lifetime Value        9134 non-null   float64
 3   Response                       9134 non-null   object
 4   Coverage                       9134 non-null   object
 5   Coverage Index                 9134 non-null   int64
 6   Education                      9134 non-null   object
 7   Education Index                9134 non-null   int64
 8   Effective To Date              9134 non-null   object
 9   Employment Status              9134 non-null   object
 10  Employment Status Index        9134 non-null   int64
 11  Gender                         9134 non-null   object
 12  Income                         9134 non-null   int64
 13  Location                       9134 non-null   object
 14  Location Index                 9134 non-null   int64
 15  Marital Status                 9134 non-null   object
 16  Marital Status Index           9134 non-null   int64
 17  Monthly Premium Auto           9134 non-null   int64
 18  Months Since Last Claim        9134 non-null   int64
 19  Months Since Policy Inception  9134 non-null   int64
 20  Number of Open Complaints      9134 non-null   int64
 21  Number of Policies             9134 non-null   int64
 22  Policy Type                    9134 non-null   object
 23  Policy Type Index              9134 non-null   int64
 24  Policy                         9134 non-null   object
 25  Policy Index                   9134 non-null   int64
 26  Renew Offer Type               9134 non-null   int64
 27  Sales Channel                  9134 non-null   object
 28  Sales Channel Index            9134 non-null   int64
 29  Total Claim Amount             9134 non-null   float64
 30  Vehicle Class                  9134 non-null   object
```

Loading [MathJax]/extensions/Safe.js

```
 31  Vehicle Class Index            9134 non-null   int64
 32  Vehicle Size                   9134 non-null   object
 33  Vehicle Size Index             9134 non-null   int64
dtypes: float64(2), int64(17), object(15)
memory usage: 2.4+ MB
```

In [15]:

```
df.isnull().sum()
```

Out[15]:

```
Customer                         0
State                            0
Customer Lifetime Value          0
Response                         0
Coverage                         0
Coverage Index                   0
Education                        0
Education Index                  0
Effective To Date                0
Employment Status                0
Employment Status Index          0
Gender                           0
Income                           0
Location                         0
Location Index                   0
Marital Status                   0
Marital Status Index             0
Monthly Premium Auto             0
Months Since Last Claim          0
Months Since Policy Inception    0
Number of Open Complaints        0
Number of Policies               0
Policy Type                      0
Policy Type Index                0
Policy                           0
Policy Index                     0
Renew Offer Type                 0
Sales Channel                    0
Sales Channel Index              0
Total Claim Amount               0
Vehicle Class                    0
Vehicle Class Index              0
Vehicle Size                     0
Vehicle Size Index               0
dtype: int64
```

In [16]:

```
df.dtypes
```

Out[16]:

```
Customer                          object
State                             object
Customer Lifetime Value          float64
Response                          object
Coverage                          object
Coverage Index                     int64
Education                         object
Education Index                    int64
Effective To Date                 object
Employment Status                 object
Employment Status Index            int64
Gender                            object
Income                             int64
Location                          object
Location Index                     int64
Marital Status                    object
Marital Status Index               int64
Monthly Premium Auto               int64
Months Since Last Claim            int64
Months Since Policy Inception      int64
Number of Open Complaints          int64
```

Loading [MathJax]/extensions/Safe.js

```
Number of Policies                int64
Policy Type                      object
Policy Type Index                 int64
Policy                           object
Policy Index                      int64
Renew Offer Type                  int64
Sales Channel                    object
Sales Channel Index               int64
Total Claim Amount              float64
Vehicle Class                    object
Vehicle Class Index               int64
Vehicle Size                     object
Vehicle Size Index                int64
dtype: object
```

In [17]:

```
df.describe()
```

Out[17]:

	Customer Lifetime Value	Coverage Index	Education Index	Employment Status Index	Income	Location Index	Marital Status Index	Month Premiu Au
count	9134.000000	9134.000000	9134.000000	9134.000000	9134.000000	9134.000000	9134.000000	9134.0000
mean	10806.669619	0.480622	1.288373	0.988395	50837.478432	0.979089	0.879790	125.8763
std	9275.806257	0.655817	1.079984	0.907454	41012.876153	0.605732	0.636838	46.4686
min	2562.310000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	82.0000
25%	5392.237500	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	92.0000
50%	7803.250000	0.000000	1.000000	1.000000	45751.000000	1.000000	1.000000	112.0000
75%	12098.922500	1.000000	2.000000	1.000000	84132.000000	1.000000	1.000000	147.0000
max	112489.260000	2.000000	4.000000	4.000000	134974.000000	2.000000	2.000000	402.0000

In [18]:

```
df.columns = df.columns.str.lower().str.replace(" ", "_")
```

In [19]:

```
df.head()
```

Out[19]:

	customer	state	customer_lifetime_value	response	coverage	coverage_index	education	education_index
0	QC35222	California	3622.69	No	Basic	0	Bachelor	2
1	AE98193	Washington	10610.21	No	Basic	0	High School or Below	0
2	TM23514	Oregon	13868.02	No	Extended	1	College	1
3	WB38524	California	4008.95	No	Basic	0	High School or Below	0
4	QZ42725	Washington	3119.69	No	Basic	0	Bachelor	2

5 rows × 34 columns

In [20]:

```
df = df.drop(columns=[
    'customer',    # just an ID
```

Loading [MathJax]/extensions/Safe.js

In [22]:

```
df.isnull().sum()
```

Out[22]:

```
state                            0
customer_lifetime_value          0
response                         0
coverage                         0
coverage_index                   0
education                        0
education_index                  0
effective_to_date                0
employment_status                0
employment_status_index          0
gender                           0
income                           0
location                         0
location_index                   0
marital_status                   0
marital_status_index             0
monthly_premium_auto             0
months_since_last_claim          0
months_since_policy_inception    0
number_of_open_complaints        0
number_of_policies               0
policy_type                      0
policy_type_index                0
policy                           0
policy_index                     0
renew_offer_type                 0
sales_channel                    0
sales_channel_index              0
total_claim_amount               0
vehicle_class                    0
vehicle_class_index              0
vehicle_size                     0
vehicle_size_index               0
dtype: int64
```

In [25]:

```
df['effective_to_date'] = pd.to_datetime(df['effective_to_date'])
df.dtypes
```

Out[25]:

```
state                                     object
customer_lifetime_value                  float64
response                                  object
coverage                                  object
coverage_index                             int64
education                                 object
education_index                            int64
effective_to_date                 datetime64[ns]
employment_status                         object
employment_status_index                    int64
gender                                    object
income                                     int64
location                                  object
location_index                             int64
marital_status                            object
marital_status_index                       int64
monthly_premium_auto                       int64
months_since_last_claim                    int64
months_since_policy_inception              int64
number_of_open_complaints                  int64
number_of_policies                         int64
policy_type                               object
policy_type_index                          int64
policy                                    object
policy_index                               int64
pe                                         int64
```

Loading [MathJax]/extensions/Safe.js

```
sales_channel                            object
sales_channel_index                       int64
total_claim_amount                      float64
vehicle_class                            object
vehicle_class_index                       int64
vehicle_size                             object
vehicle_size_index                        int64
dtype: object
```

In [26]:

```
df['has_claim'] = (df['total_claim_amount'] > 0).astype(int)
df.head()
```

Out[26]:

	state	customer_lifetime_value	response	coverage	coverage_index	education	education_index	effective_
0	California	3622.69	No	Basic	0	Bachelor	2	20
1	Washington	10610.21	No	Basic	0	High School or Below	0	20
2	Oregon	13868.02	No	Extended	1	College	1	20
3	California	4008.95	No	Basic	0	High School or Below	0	20
4	Washington	3119.69	No	Basic	0	Bachelor	2	20

5 rows × 34 columns

In [27]:

```
df['total_claim_amount'].describe()
```

Out[27]:

```
count    9134.000000
mean      586.019914
std       392.175143
min         0.130000
25%       367.545000
50%       518.325000
75%       739.147500
max      3905.870000
Name: total_claim_amount, dtype: float64
```

In [28]:

```
upper_limit = df['total_claim_amount'].quantile(0.99)
df = df[df['total_claim_amount'] <= upper_limit]
```

In [29]:

```
df['has_claim'].value_counts()
```

Out[29]:

```
1    9042
Name: has_claim, dtype: int64
```

In [37]:

```
df.tail()
```

Out[37]:

	state	customer_lifetime_value	response	coverage	coverage_index	education	education_index	effectiv
9129	California	7757.04	No	Basic	0	Bachelor	2	2
9130	California	21968.20	No	Basic	0	Bachelor	2	2
9131	California	3465.16	No	Basic	0	High School or Below	0	2
9132	California	4023.01	No	Extended	1	College	1	2

Loading [MathJax]/extensions/Safe.js

	state	customer_lifetime_value	response	coverage	coverage_index	education	education_index	effectiv
9133	California	3484.15	Yes	Basic	0	College	1	2

5 rows × 34 columns

In [39]:

```
rows_with_zero = df[df['total_claim_amount'] == 0]
rows_with_zero #everyone has a claim amount with the minimum being $0.13
```

Out[39]:

state	customer_lifetime_value	response	coverage	coverage_index	education	education_index	effective_to_dat

0 rows × 34 columns

In [41]:

```
df['has_claim'].value_counts()
```

Out[41]:

```
1    9042
Name: has_claim, dtype: int64
```

In [42]:

```
df['has_claim'].mean()
```

Out[42]:

```
1.0
```

In [43]:

```
df['total_claim_amount'].describe()
```

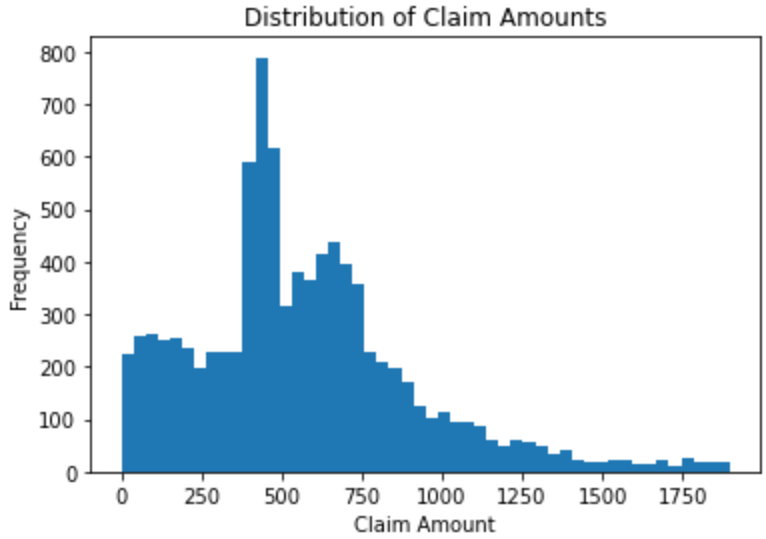
Out[43]:

```
count    9042.000000
mean      567.743655
std       346.686644
min         0.130000
25%       362.857500
50%       511.920000
75%       732.240000
max      1897.150000
Name: total_claim_amount, dtype: float64
```

In [44]:

```
import matplotlib.pyplot as plt

plt.hist(df['total_claim_amount'], bins=50)
plt.title("Distribution of Claim Amounts")
plt.xlabel("Claim Amount")
plt.ylabel("Frequency")
plt.show()
```

Loading [MathJax]/extensions/Safe.js



In [45]:
```
df.groupby('gender')['total_claim_amount'].mean()
```

Out[45]:
```
gender
F    544.515971
M    592.054387
Name: total_claim_amount, dtype: float64
```

In [46]:
```
df.groupby('coverage')['total_claim_amount'].mean()
```

Out[46]:
```
coverage
Basic       501.842916
Extended    629.150007
Premium     818.201387
Name: total_claim_amount, dtype: float64
```

In [47]:
```
df.groupby('vehicle_class')['total_claim_amount'].mean()
```

Out[47]:
```
vehicle_class
Four-Door Car     474.876576
Luxury Car       1252.627920
Luxury SUV       1259.995658
SUV               755.670815
Sports Car        739.415428
Two-Door Car      473.232911
Name: total_claim_amount, dtype: float64
```

In [48]:
```
df.groupby(pd.qcut(df['income'], 4))['total_claim_amount'].mean()
```

Out[48]:
```
income
(-0.001, 13921.0]      760.282517
(13921.0, 46355.0]     597.966420
(46355.0, 84326.0]     498.200668
(84326.0, 134974.0]    414.101952
Name: total_claim_amount, dtype: float64
```

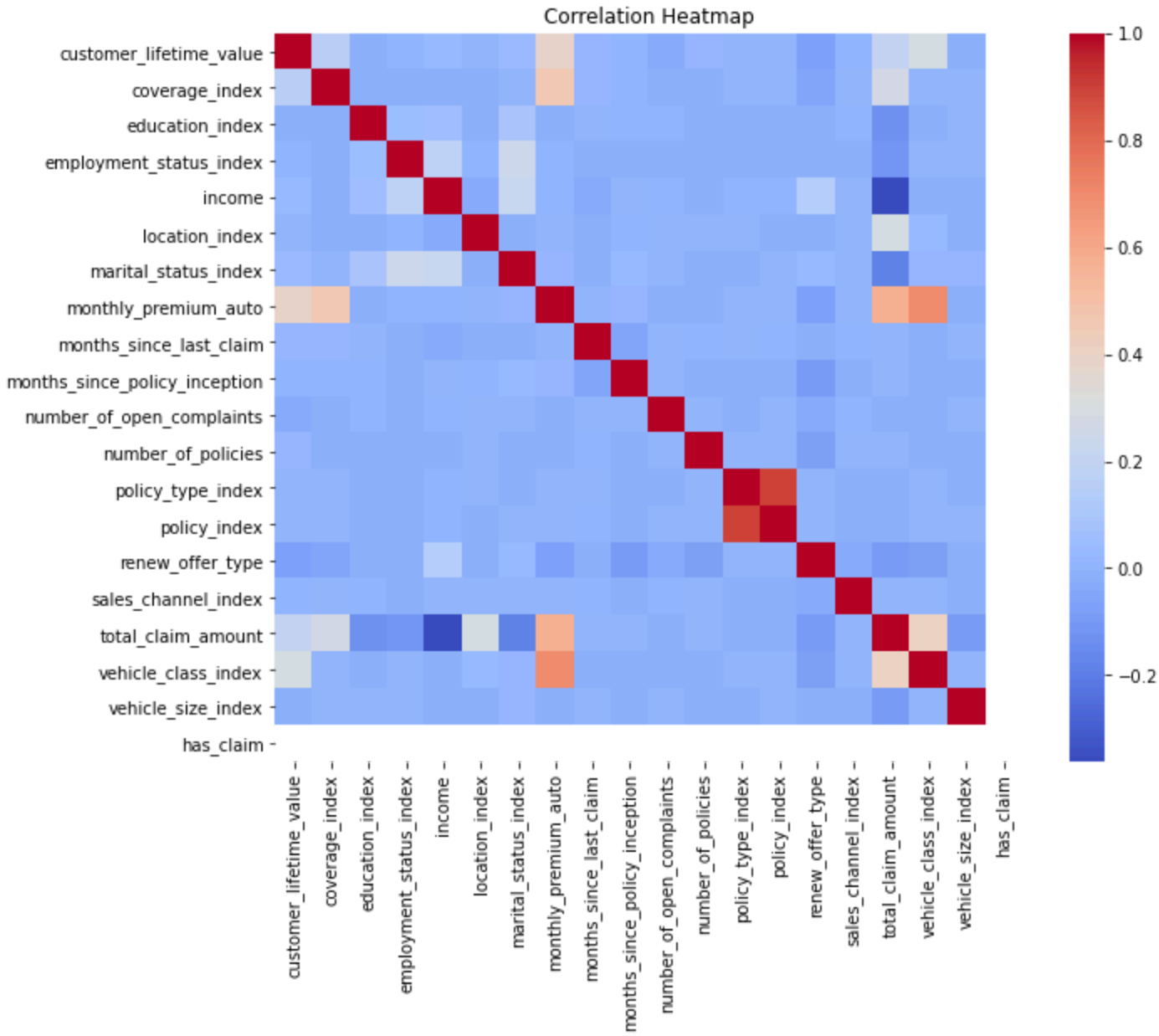
In [49]:
```
df.groupby(pd.qcut(df['months_since_last_claim'], 4))['total_claim_amount'].mean()
```

Out[49]:
```
months_since_last_claim
(-0.001, 8.0]    568.758455
(8.0, 19.0]      564.597660
(19.0, 31.0]     569.576821
(31.0, 47.0]     568.051457
Name: total_claim_amount, dtype: float64
```

Loading [MathJax]/extensions/Safe.js

In [53]:

```
import seaborn as sns

plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



In [55]:

```
df.groupby('vehicle_class').agg({
    'total_claim_amount': 'mean'
})
# SUV drivers have higher claim frequency and severity → higher risk segment
# Customers with recent claims are more likely to file again
# Higher coverage policies show higher average claim costs
```

Out[55]:

vehicle_class	total_claim_amount
Four-Door Car	474.876576
Luxury Car	1252.627920
Luxury SUV	1259.995658

Loading [MathJax]/extensions/Safe.js

vehicle_class	total_claim_amount
SUV	755.670815
Sports Car	739.415428
Two-Door Car	473.232911

In [56]:
```
df = df.drop(columns=['has_claim'])
```

In [57]:
```
df.groupby('vehicle_class')['total_claim_amount'].mean().sort_values(ascending=False)
```

Out[57]:
```
vehicle_class
Luxury SUV       1259.995658
Luxury Car       1252.627920
SUV               755.670815
Sports Car        739.415428
Four-Door Car     474.876576
Two-Door Car      473.232911
Name: total_claim_amount, dtype: float64
```

In [59]:
```
df.groupby('coverage')['total_claim_amount'].mean().sort_values(ascending=False)
```

Out[59]:
```
coverage
Premium     818.201387
Extended    629.150007
Basic       501.842916
Name: total_claim_amount, dtype: float64
```

In [61]:
```
df.groupby('employment_status')['total_claim_amount'].mean().sort_values(ascending=False)
```

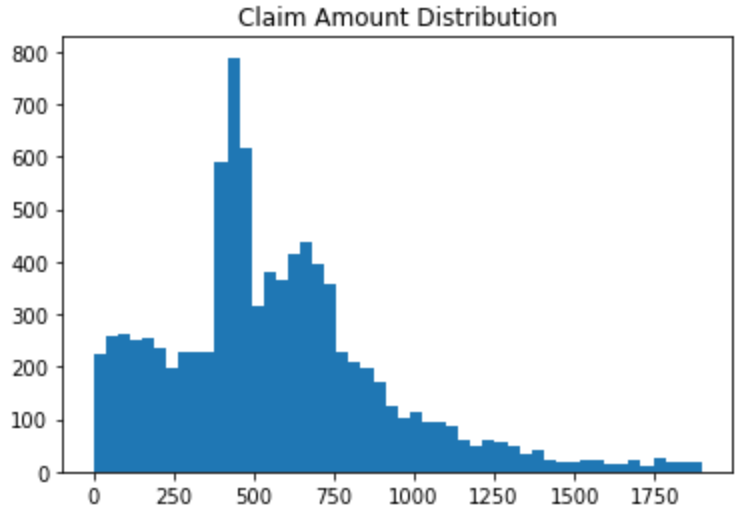
Out[61]:
```
employment_status
Unemployed       762.292949
Retired          661.366773
Medical Leave    629.526729
Disabled         629.223831
Employed         477.042463
Name: total_claim_amount, dtype: float64
```

In [62]:
```
df.groupby('vehicle_size')['total_claim_amount'].mean()
```

Out[62]:
```
vehicle_size
Large      552.498576
Medsize    549.050860
Small      643.956463
Name: total_claim_amount, dtype: float64
```

In [63]:
```
import matplotlib.pyplot as plt

plt.hist(df['total_claim_amount'], bins=50)
plt.title("Claim Amount Distribution")
plt.show()
```

Loading [MathJax]/extensions/Safe.js



In [64]:

```
df.groupby('vehicle_class').agg({
    'total_claim_amount': ['mean', 'count']
}).sort_values(('total_claim_amount', 'mean'), ascending=False)

# Luxury SUV / Luxury Car (~1250) → highest severity
# SUV / Sports Car (~740–755) → medium-high
# Standard cars (~470) → lowest
# Luxury vehicles have ~2.5× higher claim costs than standard cars
# This suggests repair/replacement costs are a major driver of severity
```

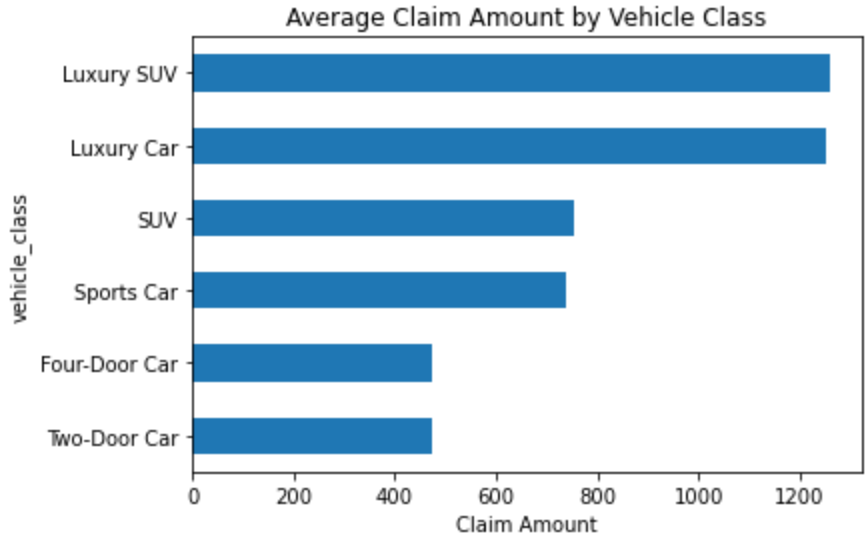
Out[64]:

	total_claim_amount	
	mean	count
vehicle_class		
Luxury SUV	1259.995658	152
Luxury Car	1252.627920	125
SUV	755.670815	1779
Sports Car	739.415428	479
Four-Door Car	474.876576	4621
Two-Door Car	473.232911	1886

In [66]:

```
import matplotlib.pyplot as plt

df.groupby('vehicle_class')['total_claim_amount'].mean().sort_values().plot(kind='barh')
plt.title("Average Claim Amount by Vehicle Class")
plt.xlabel("Claim Amount")
plt.show()
```

Loading [MathJax]/extensions/Safe.js



In [68]:

```
numeric_df = df.select_dtypes(include=['number'])
numeric_df.corr()['total_claim_amount'].sort_values(ascending=False)
```

Out[68]:

```
total_claim_amount               1.000000
monthly_premium_auto             0.574501
vehicle_class_index              0.410834
location_index                   0.287829
coverage_index                   0.275891
customer_lifetime_value          0.202635
months_since_last_claim          0.002820
number_of_policies              -0.000217
months_since_policy_inception   -0.002936
sales_channel_index             -0.003361
policy_type_index               -0.004617
policy_index                    -0.010216
number_of_open_complaints       -0.017022
vehicle_size_index              -0.087550
renew_offer_type                -0.092277
employment_status_index         -0.099903
education_index                 -0.126673
marital_status_index            -0.188951
income                          -0.359783
Name: total_claim_amount, dtype: float64
```

In [69]:

```
features = [
    'monthly_premium_auto',
    'vehicle_class_index',
    'coverage_index',
    'location_index',
    'income'
]
target = 'total_claim_amount'
```

In [70]:

```
from sklearn.model_selection import train_test_split

X = df[features]
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

In [71]:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
rain, y_train)
```

Loading [MathJax]/extensions/Safe.js

Out[71]: LinearRegression()

In [72]:
```
from sklearn.metrics import mean_squared_error, r2_score

y_pred = model.predict(X_test)

print("R^2:", r2_score(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred, squared=False))
```

```
R^2: 0.5576422278827419
RMSE: 233.16087059776984
```

In [73]:
```
coefficients = pd.DataFrame({
    'Feature': features,
    'Coefficient': model.coef_
})
coefficients
```

Out[73]:

	Feature	Coefficient
0	monthly_premium_auto	4.442949
1	vehicle_class_index	1.776788
2	coverage_index	10.998245
3	location_index	160.917539
4	income	-0.003051

In [74]:
```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print("R^2:", r2_score(y_test, y_pred_rf))
```

```
R^2: 0.8077319240384745
```

In [75]:
```
import pandas as pd

importance = pd.DataFrame({
    'Feature': features,
    'Importance': rf.feature_importances_
}).sort_values(by='Importance', ascending=False)

importance
```

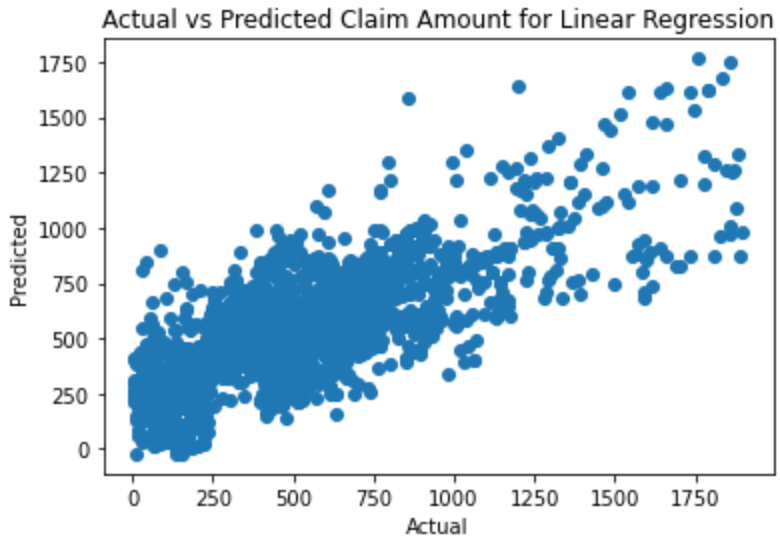
Out[75]:

	Feature	Importance
3	location_index	0.478514
0	monthly_premium_auto	0.410830
4	income	0.095657
1	vehicle_class_index	0.011976
2	coverage_index	0.003022

Loading [MathJax]/extensions/Safe.js

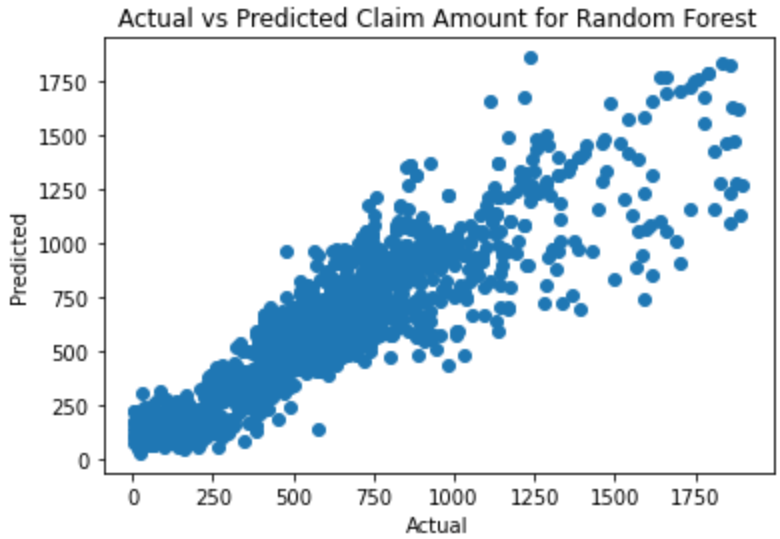
In [79]:

```
import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted Claim Amount for Linear Regression")
plt.show()
```



In [77]:

```
import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred_rf)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted Claim Amount for Random Forest")
plt.show()
```



In [80]:

```
features2 = [
    'vehicle_class_index',
    'coverage_index',
    'location_index',
    'income'
]
X2 = df[features2]
y = df['total_claim_amount']
from sklearn.model_selection import train_test_split
```

Loading [MathJax]/extensions/Safe.js

```
X2_train, X2_test, y_train, y_test = train_test_split(
    X2, y, test_size=0.2, random_state=42
)
```

In [81]:

```
from sklearn.ensemble import RandomForestRegressor

rf2 = RandomForestRegressor(random_state=42)
rf2.fit(X2_train, y_train)
```

Out[81]:

```
RandomForestRegressor(random_state=42)
```

In [82]:

```
from sklearn.metrics import r2_score, mean_squared_error

y_pred_rf2 = rf2.predict(X2_test)

print("R^2:", r2_score(y_test, y_pred_rf2))
print("RMSE:", mean_squared_error(y_test, y_pred_rf2, squared=False))
```

```
R^2: 0.7801517085991749
RMSE: 164.37296327412014
```

In [83]:

```
import pandas as pd

importance2 = pd.DataFrame({
    'Feature': features2,
    'Importance': rf2.feature_importances_
}).sort_values(by='Importance', ascending=False)

importance2
```

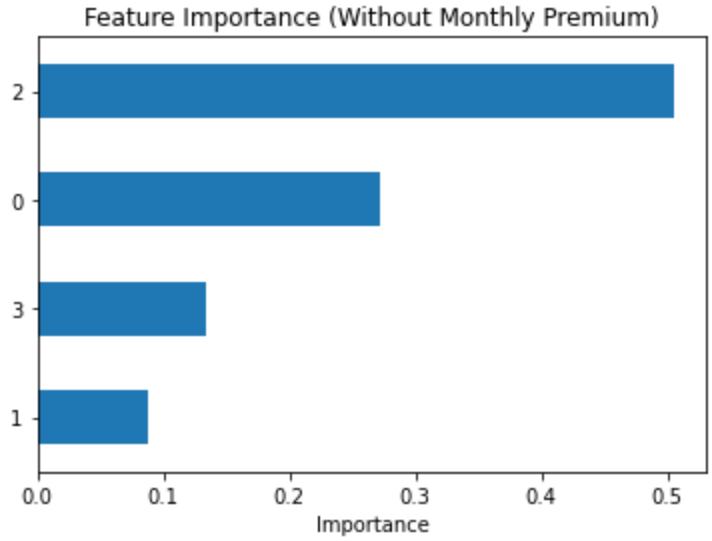
Out[83]:

	Feature	Importance
2	location_index	0.505986
0	vehicle_class_index	0.272626
3	income	0.133570
1	coverage_index	0.087818

In [85]:

```
import matplotlib.pyplot as plt

importance2.sort_values(by='Importance').plot(
    kind='barh', legend=False
)
plt.title("Feature Importance (Without Monthly Premium)")
plt.xlabel("Importance")
plt.show()
```

Loading [MathJax]/extensions/Safe.js



Loading [MathJax]/extensions/Safe.js